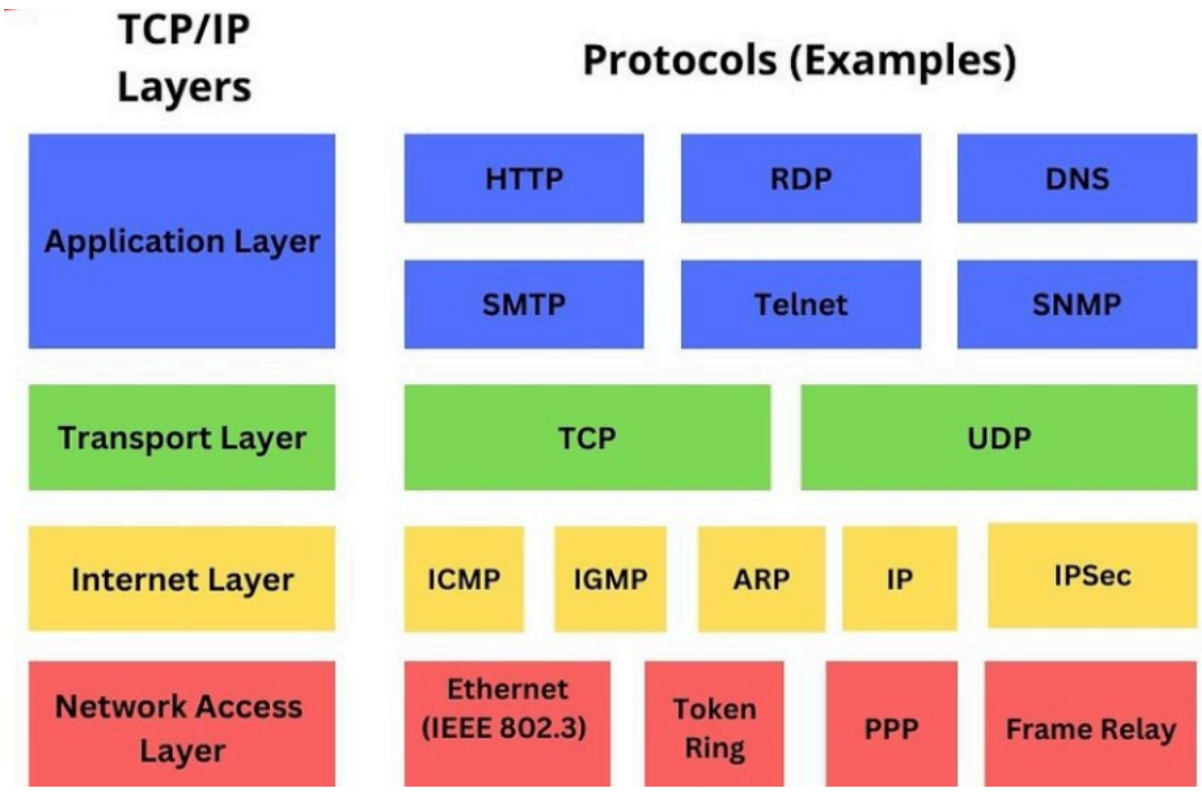
4.TCP – IP Protocol

Why the Model Exists

- Two machines communicating have multiple independent problems to solve – signal transmission, addressing, reliable delivery, application routing
- TCP/IP breaks these into 4 layers, each solving one category of problem
- Each layer only cares about what it receives and what it passes up or down – this is layered abstraction
- Same principle as your app code not caring how the database writes to disk



TCP/IP Layers

Layer 1 – Network Access Layer

- Handles physical transmission between directly connected devices – same local network, same cable, same wifi
- Deals with how bits become electrical signals, light pulses, or radio waves

MAC Address

- 48-bit address hardcoded into every network interface card at manufacture
- Example: 00:1A:2B:3C:4D:5E
- Used only for local delivery – machine to machine on the same network
- Does not travel across the internet

Ethernet

- Protocol that defines how data is packaged into frames for local wired networks
- A frame contains: source MAC, destination MAC, payload data, error-checking field

ARP (Address Resolution Protocol)

- Your machine knows the destination IP but not the MAC address
- ARP broadcasts on the local network: "who has IP 192.168.1.5, give me your MAC"
- The matching machine responds with its MAC
- Now the frame can be delivered locally

Wi-Fi (802.11)

- Same concept as Ethernet but wireless
- Frame structure slightly different to handle signal interference

What this layer does NOT do

- Does not route across the internet
- Only handles the immediate next hop

Layer 2 — Internet Layer

- Handles routing — getting a packet from your machine to any machine anywhere in the world across multiple networks

IP Address

- Unlike MAC, IP addresses are logical and can change
- IPv4 is 32 bits written as four numbers: 142.250.195.78
- IPv6 is 128 bits

What IP Does

- Wraps data from transport layer into a packet
- Packet contains: source IP, destination IP, TTL, payload
- IP is connectionless and unreliable by design — no guarantee of delivery, ordering, or fixed path

Routing

- Packet leaves your machine and goes to your router
- Router reads destination IP, checks routing table, forwards to next router
- This repeats at every router — each one is called a hop
- Typically 15 to 20 hops before reaching destination
- Different packets from the same request can take different routes

TTL (Time To Live)

- Every packet starts with a TTL value — usually 64 or 128
- Every router decrements it by 1
- If it hits 0, packet is dropped and ICMP message is sent back to source
- Prevents packets from looping forever on the internet

ICMP (Internet Control Message Protocol)

- Used for diagnostic and error messaging, not data transfer
- ping uses ICMP
- traceroute uses ICMP — exploits the TTL decrement to map each hop
- When a router drops a packet due to TTL, it sends ICMP "time exceeded" back

Subnetting

- IP address split into network portion and host portion
- Subnet mask defines where the split is
- 192.168.1.0/24 means first 24 bits = network, last 8 bits = host
- Routers use this to decide if packet is local or needs to go out

NAT (Network Address Translation)

- Your home router has one public IP from the ISP
- All your devices have private IPs (192.168.x.x)
- When packet goes out, router rewrites source IP to its public IP and records the mapping
- When response comes back, router rewrites destination IP back to your device's private IP

Layer 3 — Transport Layer

- Internet layer delivers a packet to a machine
- But a machine runs dozens of apps simultaneously — browser, Spotify, SSH
- Transport layer's job: route incoming data to the correct application using ports

Ports

- 16-bit number (0 to 65535) identifying a specific process on a machine
- Your browser opens a random port like 52341
- It connects to Google's port 443
- IP + port = socket
- OS uses the socket to deliver data to the right application

TCP — Transmission Control Protocol

- Connection-oriented, reliable, ordered, flow-controlled

Three-Way Handshake

- Before any data flows, connection is established
- Client sends SYN — "I want to connect, my sequence starts at X"
- Server sends SYN-ACK — "Acknowledged X, my sequence starts at Y"
- Client sends ACK — "Acknowledged Y"
- One full round trip before data can flow

Sequence Numbers

- Every byte sent has a sequence number
- Receiver uses this to detect missing packets and reorder out-of-order arrivals
- If packet 3 arrives before packet 2, TCP holds packet 3 and waits

Acknowledgements and Retransmission

- Receiver sends ACKs telling sender how much data has been received
- If sender does not get ACK within timeout, it retransmits
- This is how TCP guarantees delivery

Flow Control (Receive Window)

- Receiver tells sender how much buffer space it has
- Sender cannot send more than that window size at once
- Prevents sender from overwhelming receiver

Congestion Control

- TCP detects network congestion and slows down
- Slow start: begins by sending small number of packets, doubles rate every round trip
- On packet loss, backs off significantly
- Algorithms: slow start, congestion avoidance, fast retransmit

Connection Teardown

- Four-way handshake to close: FIN, ACK, FIN, ACK
- Both sides independently close their half

Cost of TCP

- All this reliability adds overhead
- Handshake alone adds minimum one round trip before data flows
- Head-of-line blocking — if one packet is lost, everything behind it stalls

UDP — User Datagram Protocol

- Connectionless and unreliable
- No handshake, no acknowledgements, no ordering, no retransmission
- Just fires packets and forgets

Why use something unreliable

- Reliability has cost — latency, overhead, head-of-line blocking
- For some apps a lost packet is better than a delayed one

Use Cases

- Video calls — lost frame is better than a frozen call waiting for retransmit
- DNS — small single request-response, TCP overhead not worth it
- Online gaming — stale position packet is useless, newer one already arrived
- Live streaming — better to drop a packet than pause playback

QUIC

- UDP is the base for QUIC, which HTTP/3 is built on

- QUIC reimplements reliability at the application layer with far less overhead than TCP
- Each stream is independent — lost packet in one stream does not block others

Layer 4 — Application Layer

- Lower layers handle delivery
- Application layer defines the format and meaning of messages between applications

HTTP / HTTPS

- Defines structure of web requests and responses
- Request has: method (GET, POST, etc.), headers, optional body
- Response has: status code, headers, body
- HTTPS is HTTP with TLS encryption on top

DNS

- Translates domain names to IP addresses
- Query goes to DNS resolver, which queries root servers, TLD servers, authoritative servers in hierarchy
- Result is cached with a TTL
- Uses UDP by default (small, fast, single exchange)

TLS

- Sits between Transport and Application
- Handles encryption, identity verification via certificates, data integrity
- TLS handshake: exchange certificates, verify identity, negotiate shared symmetric key
- All subsequent data encrypted with that symmetric key

WebSocket

- Built on top of TCP
- Starts with an HTTP handshake, then connection is upgraded to WebSocket
- Full-duplex — both sides can send messages any time without request-response cycle
- Used in Google Docs, chat apps, live collaboration tools

HTTP/2

- Multiple requests multiplexed over a single TCP connection using streams
- Adds header compression
- Still suffers from TCP head-of-line blocking — one lost packet stalls all streams

HTTP/3

- Replaces TCP with QUIC (UDP-based)
- Each stream is independent — lost packet does not block other streams
- Faster connection setup — supports 0-RTT in some cases

OSI vs TCP/IP

- OSI has 7 layers — theoretical reference model
- TCP/IP has 4 layers — what is actually implemented in the real world

Mapping

- OSI Layer 7 Application → TCP/IP Application Layer
- OSI Layer 6 Presentation (encryption, encoding) → absorbed into TCP/IP Application Layer
- OSI Layer 5 Session (session management) → absorbed into TCP/IP Application Layer
- OSI Layer 4 Transport → TCP/IP Transport Layer
- OSI Layer 3 Network → TCP/IP Internet Layer
- OSI Layer 2 Data Link → TCP/IP Network Access Layer
- OSI Layer 1 Physical → TCP/IP Network Access Layer

Why OSI Still Matters

- Engineers use OSI layer numbers as vocabulary

- Layer 7 load balancer — reads and routes based on HTTP content (application-level)
- Layer 4 load balancer — routes based on IP and TCP port, does not read HTTP
- You will hear this constantly in system design interviews and production discussions

Data Flow — Encapsulation and Decapsulation

Sending (Encapsulation — each layer wraps the one above)

- Application Layer creates HTTP request
- Transport Layer wraps it in TCP segment — adds source port, destination port, sequence number
- Internet Layer wraps in IP packet — adds source IP, destination IP, TTL
- Network Access Layer wraps in Ethernet frame — adds source MAC, destination MAC

Receiving (Decapsulation — each layer unwraps its own header)

- Ethernet frame arrives, MAC checked, IP packet extracted
- IP packet checked, destination IP matched, TCP segment extracted
- TCP segment checked, port matched to correct application, data passed up
- Application reads the HTTP request or WebSocket message

Important detail about MACs

- IP addresses stay the same end to end across the entire journey
- MAC addresses are rewritten at every single router hop
- Source MAC becomes the router's outgoing MAC, destination MAC becomes the next router's MAC
- Because MAC is only for local delivery between neighbors

Real Life — Google Docs

Setup

- You open Google Docs, Chrome resolves docs.google.com via DNS over UDP
- TCP connection is established with Google's server (three-way handshake)
- TLS handshake happens over that TCP connection — certificate verified, symmetric key negotiated
- HTTP upgrade request sent — connection becomes a WebSocket

You type a character

- Application Layer: Google Docs JS packages your keystroke as a WebSocket message (operational transform / delta), handed to OS
- Transport Layer: OS wraps in TCP segment — source port 54231, destination port 443, sequence number assigned
- Internet Layer: Wrapped in IP packet — source IP your machine, destination IP Google's server, TTL set
- Network Access Layer: Destination IP is not local, so packet goes to router — ARP gets router's MAC, Ethernet frame created with your MAC as source and router's MAC as destination

On the wire

- Your router receives frame, strips Ethernet header, reads IP packet
- Rewrites Ethernet frame with new MACs for next hop, forwards
- Repeats across 15-20 routers until Google's data center

At Google's server

- Ethernet frame arrives, decapsulated upward layer by layer
- WebSocket message extracted and handed to Google Docs backend
- Server processes the keystroke, updates document state
- Response goes back through the same process in reverse

TLS throughout

- All WebSocket data is encrypted
- Lower layers see only encrypted bytes
- Encryption and decryption handled transparently between Transport and Application